Amazon SDE Intern - DP Cheatsheet

8 Problems | Both Solutions | Interview Ready

Interview Strategy

Question Flow (15-20 min)

1. **Clarify** (30s): Constraints, edge cases
2. **Example** (30s): Verify with test case
3. **Brute Force** (30s): Mention recursive
4. **DP Approach** (1-2min): State, transition
5. **Code** (3-4min): DP array version
6. **Trace** (1-2min): Walk through example
7. **Optimize** (1min): "Can use O(1) space"
8. **Complexity** (30s): Both versions

Key Phrases

- "Classic DP with overlapping subproblems"
- "dp[i] represents..."
- "Can optimize space to O(1) using two variables"
- "Only need last 2/previous row values"

1. House Robber (LC 198)

Problem: Rob houses, maximize money, no adjacent.
State: dp[i] = max money from houses 0 to i
Transition: dp[i] = max(dp[i-1], nums[i] + dp[i-2])
Base: dp[0] = nums[0], dp[1] = max(nums[0], nums[1])

Solution 1: DP Array - O(n) space

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        int skip = dp[i-1];
        int rob = nums[i] + dp[i-2];
        dp[i] = Math.max(skip, rob);
    }
    return dp[nums.length - 1];
}
```

Solution 2: Space Optimized - O(1) space

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int prev2 = nums[0];
    int prev1 = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        int current = Math.max(prev1,
                               nums[i] + prev2);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Time: O(n) **Space:** O(n) → O(1)
Example: [2,7,9,3,1] → 12 (rob 0,2,4)
Key: Only need last 2 values, use prev1/prev2

2. Min Cost Climbing Stairs (LC 746)

Problem: Climb stairs with costs, 1 or 2 steps.
State: dp[i] = min cost to reach position i
Transition: dp[i] = min(cost[i-1]+dp[i-1], cost[i-2]+dp[i-2])
Base: dp[0] = 0, dp[1] = 0 (start free)

Solution 1: DP Array - O(n) space

```
public int minCostClimbingStairs(int[] cost) {
    int[] dp = new int[cost.length + 1];
    dp[0] = 0;
    dp[1] = 0;

    for (int i = 2; i <= cost.length; i++) {
        int take1 = cost[i-1] + dp[i-1];
        int take2 = cost[i-2] + dp[i-2];
        dp[i] = Math.min(take1, take2);
    }
    return dp[cost.length];
}
```

Solution 2: Space Optimized - O(1) space

```
public int minCostClimbingStairs(int[] cost) {
    int prev2 = 0;  // dp[i-2]
    int prev1 = 0;  // dp[i-1]

    for (int i = 2; i <= cost.length; i++) {
        int current = Math.min(
            cost[i-1] + prev1,
            cost[i-2] + prev2
        );
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Time: O(n) **Space:** O(n) → O(1)
Key: Pay cost[i] to LEAVE step i, not arrive

3. Stock I (LC 121)

Problem: One transaction, maximize profit.
Approach: Track min price, maximize (current - min)

Solution - O(1) space

```
public int maxProfit(int[] prices) {
    int maxProfit = 0;
    int minPrice = prices[0];

    for (int price : prices) {
        minPrice = Math.min(minPrice, price);
        maxProfit = Math.max(maxProfit,
                             price - minPrice);
    }
    return maxProfit;
}
```

Time: O(n) **Space:** O(1)
Key: Buy at lowest, sell at highest after

4. Stock II (LC 122)

Problem: Unlimited transactions, max profit.
Approach: Capture every upward movement

Solution - O(1) space

```
public int maxProfit(int[] prices) {
    int profit = 0;

    for (int i = 1; i < prices.length; i++) {
        if (prices[i] > prices[i-1]) {
            profit += prices[i] - prices[i-1];
        }
    }
    return profit;
}
```

Time: O(n) **Space:** O(1)
Why: (b-a)+(c-b)+(d-c) = d-a (telescoping)

5. Coin Change (LC 322)

Problem: Min coins for amount, unlimited use.
State: dp[i] = min coins for amount i
Transition: dp[i] = min(dp[i], 1 + dp[i-coin])
Base: dp[0] = 0

Solution - O(amount) space

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); // infinity
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin) {
                dp[i] = Math.min(dp[i],
                                 1 + dp[i - coin]);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Time: O(amount × coins) **Space:** O(amount)
Key: Forward iteration (allows reuse), amount+1 = infinity
Note: Can't optimize to O(1) - need all previous amounts

6. 0/1 Knapsack

Problem: Max value, weight ≤ W, each item once.
State: dp[i][w] or dp[w] = max value
Transition: max(skip item, take item)

Solution 1: 2D DP - O(n×W) space

```
public int knapsack(int W, int val[], int wt[]) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];

    // Base case: dp[0][w] = 0 (no items)

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i-1] > w) {
                // Item too heavy, skip
                dp[i][w] = dp[i-1][w];
            } else {
                // Choose max(skip, take)
                dp[i][w] = Math.max(
                    dp[i-1][w],  // skip
                    val[i-1] + dp[i-1][w-wt[i-1]] // take
                );
            }
        }
    }
    return dp[n][W];
}
```

Solution 2: 1D DP - O(W) space

```
public int knapsack(int W, int val[], int wt[]) {
    int[] dp = new int[W + 1];

    for (int i = 0; i < wt.length; i++) {
        // BACKWARD iteration (important!)
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = Math.max(
                dp[w],                  // skip
                val[i] + dp[w - wt[i]]  // take
            );
        }
    }
    return dp[W];
}
```

Time: O(n × W) **Space:** O(n×W) → O(W)
Why backward? Forward allows reusing same item
Key: In 2D, we look at previous row (i-1). In 1D, backward ensures we use "old" values (= previous row)

7. Longest Common Subsequence (LC 1143)

Problem: Longest subsequence in both strings.
State: dp[i][j] = LCS of text1[0..i-1], text2[0..j-1]
Transition:

- Match: dp[i][j] = 1 + dp[i-1][j-1]
- No match: max(dp[i-1][j], dp[i][j-1])

Solution 1: 2D DP - O(m×n) space

```
public int longestCommonSubsequence(
        String text1, String text2) {
    int m = text1.length();
    int n = text2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i-1) ==
                text2.charAt(j-1)) {
                dp[i][j] = 1 + dp[i-1][j-1];
            } else {
                dp[i][j] = Math.max(
                    dp[i-1][j],   // skip from text1
                    dp[i][j-1]    // skip from text2
                );
            }
        }
    }
    return dp[m][n];
}
```

Solution 2: Space Optimized - O(n) space

```
public int longestCommonSubsequence(
        String text1, String text2) {
    int m = text1.length();
    int n = text2.length();

    int[] prev = new int[n + 1];
    int[] curr = new int[n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i-1) ==
                text2.charAt(j-1)) {
                curr[j] = 1 + prev[j-1];
            } else {
                curr[j] = Math.max(prev[j],
                                   curr[j-1]);
            }
        }
        // Swap arrays
        int[] temp = prev;
        prev = curr;
        curr = temp;
    }
    return prev[n];
}
```

Time: O(m × n) **Space:** O(m×n) → O(min(m,n))
Example: "abcde", "ace" → 3
Key: Only need previous row, use two arrays

8. Unique Paths (LC 62)

Problem: Paths in m×n grid, right/down only.
State: dp[i][j] = paths to reach (i,j)
Transition: dp[i][j] = dp[i-1][j] + dp[i][j-1]

Solution 1: 2D DP - O(m×n) space

```
public int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];

    // Initialize first row and column to 1
    for (int[] row : dp) {
        Arrays.fill(row, 1);
    }

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }
    return dp[m-1][n-1];
}
```

Solution 2: Space Optimized - O(n) space

```
public int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1); // First row all 1s

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] = dp[j] + dp[j-1];
            //   from top  from left
        }
    }
    return dp[n-1];
}
```

Time: O(m × n) **Space:** O(m×n) → O(n)
Key: dp[j] = old value (top) + dp[j-1] (left)

9. Longest Increasing Subsequence (LC 300)

Problem: Length of longest increasing subsequence.
State: dp[i] = LIS length ENDING at i
Transition: For j < i where nums[j] < nums[i]:
dp[i] = max(dp[i], dp[j] + 1)

Solution - O(n) space (Can't optimize further)

```
public int lengthOfLIS(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    int maxLength = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i],
                                 dp[j] + 1);
            }
        }
        maxLength = Math.max(maxLength, dp[i]);
    }
    return maxLength;
}
```

Time: O(n^2) **Space:** O(n)
Key: Need all dp values, can't optimize to O(1)
Check ALL previous j, not just consecutive!
Example: [10,9,2,5,3,7,101,18] → 4
Note: O(n log n) solution exists with binary search but O(n^2) sufficient for interviews

Space Optimization Summary

Problem	Original	Optimized
House Robber	O(n)	O(1)
Min Cost Stairs	O(n)	O(1)
Stock I/II	O(1)	O(1)
Coin Change	O(amt)	O(amt)
Knapsack	O(n×W)	O(W)
LCS	O(m×n)	O(min(m,n))
Unique Paths	O(m×n)	O(n)
LIS	O(n)	O(n)

When Space Optimization Works:

- Only need last 1-2 values (linear DP)
- Only need previous row (2D DP)
- Need to look back at multiple previous values

Interview Strategy for Both Solutions

Recommended Approach:

1. Start with DP Array (Clearer):

- Easier to explain
- Easier to debug
- Shows understanding of DP

2. Then Mention Optimization:

Say: "I can optimize space from O(n) to O(1) by using just two variables since we only need the last two values."

3. Code Optimized If Asked:

- Show sliding window pattern
- Explain prev1/prev2 concept
- Demonstrate space savings

When to Skip Optimization:

- Time is running short
- Interviewer says "this is fine"
- Problem is already complex

Pattern Recognition

See	Think
"maximize/minimize"	DP or Greedy
"number of ways"	DP (counting)
"can you/possible"	DP (boolean)
"subsequence"	Skip OK, order
"subarray"	Consecutive only
"at most k"	State machine
"unlimited"	Unbounded/Greedy
"exactly once"	0/1 knapsack
"two strings"	2D DP
"grid/matrix"	2D DP, paths

Edge Cases Checklist

- Empty input: [] → 0
- Single element: [x] → handle separately
- All same: [3,3,3] → test
- All increasing: [1,2,3,4] → optimal
- All decreasing: [4,3,2,1] → worst
- Two elements: minimum comparison
- Negatives/zeros: handle if allowed

Debugging Checklist

If solution fails:

1. Array bounds valid? (i-1, i+1)
2. Base cases initialized?
3. Loop direction correct? (forward vs backward)
4. Using i vs i-1 properly?
5. Trace smallest example by hand
6. Print dp array values
7. Return dp[n] or dp[n-1]?
8. In space-optimized: updating prev1/prev2 correctly?

Complexity Quick Reference

Problem	Time	Space
House Robber	O(n)	O(1)
Min Cost Stairs	O(n)	O(1)
Stock I	O(n)	O(1)
Stock II	O(n)	O(1)
Coin Change	O(a×c)	O(a)
Knapsack	O(n×W)	O(W)
LCS	O(m×n)	O(n)
Unique Paths	O(m×n)	O(n)
LIS	O(n^2)	O(n)

a=amount, c=coins, n=array length

Final Tips

Day Before:

- Review this cheatsheet
- Code 2-3 problems from scratch
- Practice explaining both versions
- Sleep 8+ hours

During Interview:

- Think out loud constantly
- Draw examples/diagrams
- Start with DP array version
- Mention space optimization
- Accept hints gracefully

Red Flags to Avoid:

- Coding in silence
- Jumping to optimized without explaining
- Refusing to explain approach
- Ignoring interviewer hints

You're Ready!

Both Solutions Covered!
Trust Your Preparation!